Deep Reinforcement Learning Comparison

Neural Networks & Deep Learning – 2025

Scuola Superiore Sant'Anna, Pisa

Instructor: Prof. Giorgio Buttazzo

Dimitrios Tsiakmakis*

PhD Candidate, Biorobotics Institute

`dimitrios.tsiakmakis@santannapisa.it`

Google Scholar Profile

April 3, 2026

Abstract

This study compares three deep reinforcement learning algorithms (Proximal Policy Optimization (PPO), Soft Actor-Critic (SAC), and Twin Delayed Deep Deterministic Policy Gradient (TD3)) on the `HalfCheetah-v5` continuous control benchmark. All methods were implemented via Stable-Baselines3 with RL-Zoo tuned hyperparameters and trained for 10^6 timesteps across 3 random seeds. SAC achieved the highest mean reward ($12{,}198.7 \pm 511.3$), followed by TD3 ($11{,}014.8 \pm 495.1$), while PPO obtained a substantially lower mean reward ($2{,}358.9 \pm 1{,}450.4$), reflecting the inherent sample inefficiency of on-policy methods within the given training budget. Off-policy methods proved decisively more sample-efficient, with TD3 offering the best cost-to-performance ratio and SAC yielding marginally higher rewards at roughly $1.5\times$ the computational cost. A video demonstration of the best trained policies is available at: `https://youtube.com/shorts/qsyHMhZ3jCQ`. The code is available on: GitHub

*The writing of this report was assisted by Claude Sonnet 4.6 (Anthropic) for grammar and phrasing, and by GitHub Copilot (Microsoft) for code editing.

1 Introduction

Reinforcement learning in computer science is a general term that has been given to a family of techniques in which the learning system attempts to learn through direct interaction with the environment. It is widely accepted and applied to robot motion control, task optimization, etc. The concept of reinforcement learning is inspired by the corresponding analogues of reward and punishment learning found as learning models of living beings (introduced by psychologists '70). The goal of the learning system is to maximize a function of the numerical reinforcement signal (reward), for example the expected value of the reinforcement signal at the next step. The system is not guided by an external supervisor as to which action it should take, but must discover for itself which actions are the ones that will yield the greatest gain.

1.1 Proximal Policy Optimization (PPO)

PPO [6] modifies standard policy gradient implementations by substituting the loss function with L^{CLIP} or L^{KLPEN}. When the policy and value function share parameters, the objective is augmented to include value function error and an entropy bonus for exploration. This results in the following maximization objective:

$$L_t^{CLIP+VF+S}(\theta) = \hat{E}_t[L_t^{CLIP}(\theta) - c_1 L_t^{VF}(\theta) + c_2 S[\pi_\theta](s_t)] \tag{1}$$

where c_1 and c_2 are coefficients, S is the entropy bonus, and L_t^{VF} represents the squared error $(V_\theta(s_t) - V_t^{targ})^2$.

For advantage estimation, the algorithm utilizes a truncated Generalized Advantage Estimation (GAE) over fixed-length trajectory segments of size T. The estimator is defined as:

$$\hat{A}_t = \delta_t + (\gamma\lambda)\delta_{t+1} + \cdots + (\gamma\lambda)^{T-t+1}\delta_{T-1} \tag{2}$$

where $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$. The algorithm proceeds by collecting T timesteps across N parallel actors and optimizing the surrogate loss over K epochs using Adam.

1.2 Soft Actor-Critic (SAC)

SAC[4] is an off-policy deep reinforcement learning algorithm based on the maximum entropy framework, designed to address high sample complexity and brittle convergence. By training the actor to maximize both expected reward and policy entropy, SAC encourages robust exploration and training stability:

$$J(\pi) = \sum_{t=0}^{T} \mathbb{E}_{(s_t,a_t)\sim\rho_\pi}[r(s_t, a_t) + \alpha\mathcal{H}(\pi(\cdot|s_t))] \tag{3}$$

The algorithm alternates between soft policy evaluation and improvement by concurrently optimizing three parameterized networks. First, the state value network (V_ψ) minimizes the squared residual error relative to the soft Q-value. Second, the critic network (Q_θ) minimizes the soft Bellman residual, explicitly utilizing the minimum of two independent Q-networks to mitigate positive overestimation bias. Finally, the actor network (π_ϕ) is updated by minimizing the expected KL-divergence against the exponential of the Q-function, employing the reparameterization trick to yield a lower-variance gradient estimator.

1.3 Twin Delayed Deep Deterministic policy gradient (TD3)

TD3[3] improves upon DDPG[5] by addressing function approximation errors to enhance stability. The algorithm maintains two critic networks and one actor. It mitigates overestimation by updating critics toward the minimum target value of actions, which include added clipped noise ϵ:

$$y = r + \gamma \min_{i=1,2} Q_{\theta_i'}(s', \pi_{\phi'}(s') + \epsilon) \tag{4}$$

Furthermore, TD3 employs delayed policy updates, modifying the actor only every d iterations relative to the critic updates.

1.4 On-policy vs off-policy algorithms

A fundamental distinction in reinforcement learning lies between **on-policy** and **off-policy** methods. On-policy algorithms, such as PPO, learn exclusively from experience generated by the current version of the policy: after each batch of environment interactions is collected and used for a gradient update, the data is discarded and the process repeats. This ensures theoretical consistency between the data distribution and the policy being optimized, but at the cost of sample efficiency, where every environment interaction is used only once. Off-policy algorithms, such as SAC and TD3, relax this constraint by storing all past transitions in a *replay buffer*, regardless of which policy generated them. At each training step, a random mini-batch is sampled from the buffer and used to update the current policy, allowing each experience to be reused across many gradient updates. This decoupling of data collection from policy optimization leads to substantially greater sample efficiency, enabling off-policy methods to extract far more learning signal from the same number of environment interactions, where is a key advantage on complex continuous control tasks.

2 Methodology and Experimental Setup

2.1 MuJoCo: Advanced physics simulation

MuJoCo was selected as the simulation platform for our experiments due to its advanced physics capabilities, though alternative open-source simulators such as Webots or NVIDIA Isaac Sim could equally be employed.

MuJoCo is a free, open-source physics engine designed to support research and development across disciplines including robotics, biomechanics, graphics, and animation, wherever fast and accurate simulation is required. Unlike conventional simulators, MuJoCo was designed from the ground up with model-based optimization as its core objective, placing particular focus on contact dynamics. This makes it well-suited for scaling computationally demanding methods such as optimal control, state estimation, system identification, and automated mechanism design to complex contact-rich systems. Beyond optimization-focused applications, MuJoCo also supports more traditional use cases, including pre-deployment validation of control algorithms, interactive scientific visualization, and animation and gaming applications.

2.2 Benchmark Environment

All experiments were conducted on the `HalfCheetah-v5` continuous control task from the Gymnasium MuJoCo suite[1]. HalfCheetah is a planar bipedal robot with 6 actuated joints, an 17-dimensional observation space (joint positions and velocities), and a 6-dimensional continuous action space (joint torques). The reward function incentivizes forward locomotion velocity while penalizing control effort, making it a standard and challenging benchmark for continuous-action deep RL algorithms. Episodes were capped at 1000 timesteps via a `TimeLimit` wrapper.

2.3 Algorithms and Implementations

Three state-of-the-art actor-critic algorithms were compared: PPO [6], SAC [4] and TD3 [3]. All algorithms were implemented using `Stable-Baselines3`[2], a reliable and well-maintained PyTorch-based RL library that provides standardized, reproducible implementations of modern deep RL methods. All agents used the default `MlpPolicy`, consisting of a multi-layer perceptron with two hidden layers of 64 units each and `tanh` activations.

[1] `https://gymnasium.farama.org/environments/mujoco/`
[2] `https://stable-baselines3.readthedocs.io/en/master/`

2.4 Hyperparameters

Hyperparameters were adopted from the `rl-baselines3-zoo`[3] tuned defaults for the `HalfCheetah-v5` environment and differ per algorithm to reflect each method's known optimal operating regime, as summarized in Table 1.

Table 1: Algorithm-specific hyperparameters (RL-Zoo tuned defaults for `HalfCheetah-v4`).

Hyperparameter	**PPO**	**SAC**	**TD3**
Batch size	64	256	256
Rollout steps (T)	512	–	–
Replay buffer size	–	10^6	10^6
Learning rate α	2.06×10^{-5}	3×10^{-4}	1×10^{-3}
Discount factor γ	0.98	0.99	0.99
GAE λ	0.92	–	–
Clipping range	0.1	–	–
Entropy coefficient	4.0×10^{-4}	auto	–
Soft update τ	–	0.005	0.005
Policy delay d	–	–	2
Warm-up steps	–	10,000	10,000
Network architecture	[256, 256]	[64, 64]	[400, 300]
Epochs per update	20	1	1
Max gradient norm	0.8	–	–
Activation function	ReLU	ReLU	ReLU

2.5 Training Protocol

Each algorithm was trained for 10^6 environment timesteps per run. To account for stochastic variability, every algorithm–environment combination was repeated across 3 independent random seeds (42, 123, 456), resulting in 9 total training runs. All three algorithms used a plain `DummyVecEnv` without observation normalization, ensuring a consistent and fair comparison across on-policy and off-policy methods. In particular, `VecNormalize` was not applied to any algorithm: for off-policy methods (SAC, TD3) it would introduce non-stationarity in the replay buffer, as normalization statistics would shift while older transitions remain stored under different statistics; for PPO, omitting normalization aligns the training conditions with those of the off-policy baselines. Reward normalization was disabled for all algorithms.

2.6 Evaluation and Metrics

During training, a dedicated evaluation environment (with frozen normalization statistics synchronized from the training environment) was used to assess policy performance

[3] `https://stable-baselines3.readthedocs.io/en/master/guide/rl_zoo.html`

every 10,000 timesteps over 5 deterministic episodes, enabling learning curve tracking. Additionally, a finer-grained learning curve callback recorded performance every 5,000 timesteps. At the end of training, each policy was evaluated over 10 deterministic episodes to obtain final performance metrics: mean reward, standard deviation, median, minimum and maximum episode return, and mean episode length. Wall-clock training time and peak memory usage (RAM) were also recorded via `psutil`.

2.7 Software and Reproducibility

Experiments were implemented in Python using the following libraries: `Stable-Baselines3` for RL algorithm implementations, `Gymnasium` with the `MuJoCo` backend for environment simulation, `PyTorch` as the underlying deep learning framework, `NumPy` and `Pandas` for data handling, `SciPy` for statistical testing, and `Matplotlib/Seaborn` for visualization. TensorBoard logs were saved for each run to support training diagnostics. All random seeds were explicitly set to ensure reproducibility.

3 Results

3.1 Final Performance

Table 2 summarizes the mean episode reward and standard deviation across 3 seeds after 10^6 training timesteps on `HalfCheetah-v5`. The off-policy algorithms substantially outperformed the on-policy baseline. **SAC** achieved the *highest* mean reward of $12{,}198.7 \pm 511.3$, followed closely by **TD3** at $11{,}014.8 \pm 495.1$. **PPO** obtained a mean reward of $2{,}358.9 \pm 1{,}450.4$, which, while positive, is approximately $5\times$ lower than SAC and reflects the sample inefficiency of on-policy learning within the given timestep budget. A video demonstration of the best trained policies for each algorithm is available at: `https://youtube.com/shorts/qsyHMhZ3jCQ`

Table 2: Final evaluation results on `HalfCheetah-v5` (10^6 timesteps, 3 seeds, 10 evaluation episodes per seed).

Algorithm	Mean Reward	Std	Median	Train Time (s)	Train Time (h)
PPO	2,358.9	1,450.4	2,385.6	837 ± 21	$\sim 0{,}23$
SAC	12,198.7	511.3	12,272.4	$6{,}991 \pm 25$	$\sim 1{,}95$
TD3	11,014.8	495.1	11,010.9	$4{,}651 \pm 164$	$\sim 1{,}29$

3.2 Learning Curves

Figure 1 shows the mean episode reward throughout training, averaged across seeds with shaded standard deviation bands. Both SAC and TD3 exhibit rapid early improvement, reaching rewards above 8,000 within the first 200,000 timesteps, after which their learning

rates gradually plateau toward convergence. SAC consistently leads TD3 throughout training and maintains a clear advantage at the end of the training budget. PPO improves slowly but steadily across the full 10^6 timesteps, reaching approximately 2,400 reward by the end, suggesting it has not yet converged and would likely continue improving given additional environment interactions.

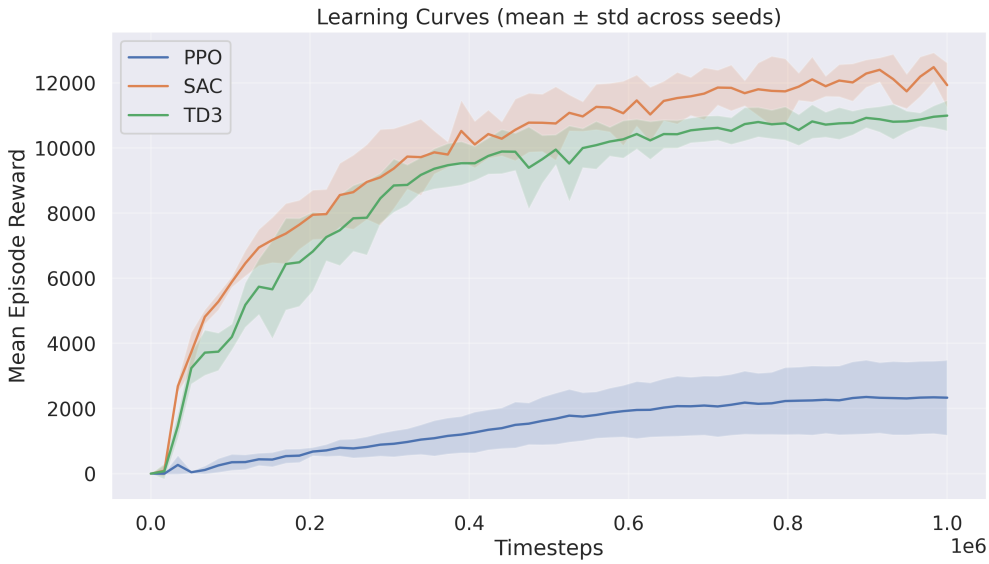

Figure 1: Learning curves on `HalfCheetah-v5`. Mean episode reward averaged over 3 seeds; shaded bands denote ±1 standard deviation.

3.3 Reward Distribution Across Seeds

Figure 2 shows the reward distribution across seeds as box plots. TD3 exhibits the tightest spread (std = 495), indicating high consistency across random initializations. SAC shows similarly low variance (std = 511), with seed 42 achieving 12,729 versus seed 456 at 11,709. PPO displays substantially higher variance (std = 1,450), with seed 42 reaching 4,032 while seeds 123 and 456 converged to only ∼1,500, revealing sensitivity to initialization that is characteristic of on-policy gradient methods[2].

3.4 Training Time

Figure 3 reports wall-clock training time. PPO was the fastest at ∼837 s (∼14 min), benefiting from CPU execution and the absence of replay buffer management. TD3 required ∼4,651 s (∼78 min), while SAC was the most computationally expensive at∼6,991 s (∼117 min). SAC is slower than TD3 because it performs more frequent and computationally heavier updates due to stochastic policy sampling, log-probability/entropy terms in both actor and critic losses and optional temperature optimization, whereas TD3 uses cheaper deterministic updates and delays actor updates.

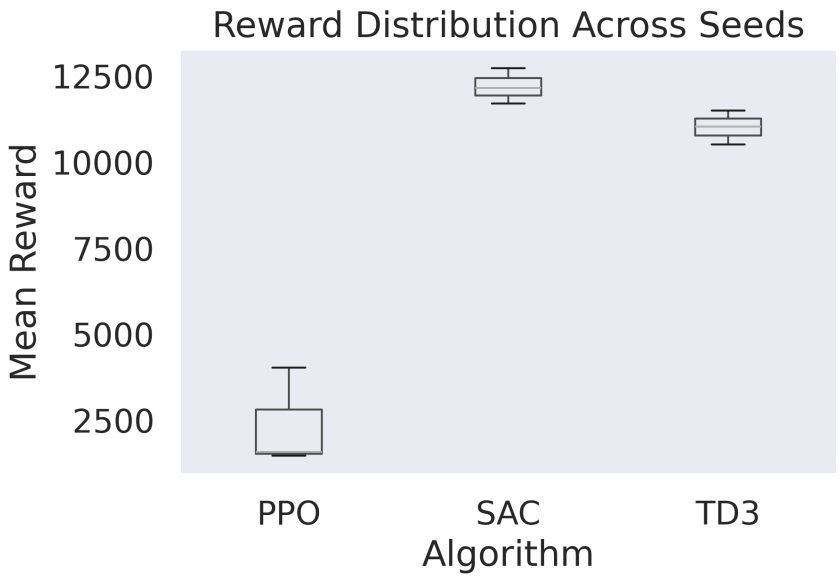

Figure 2: Reward distribution across seeds. Boxes show the interquartile range; whiskers extend to min/max values.

4 Discussion

4.1 Off-Policy vs On-Policy Sample Efficiency

The main result of this study is the substantial performance gap between the two off-policy algorithms (SAC, TD3) and the on-policy baseline (PPO). This result is consistent with the broader reinforcement learning literature and reflects a fundamental architectural difference: off-policy methods decouple data collection from policy optimization by storing all past transitions in a replay buffer, allowing each environment interaction to be reused across multiple gradient updates. PPO, by contrast, discards all collected experience after each policy update, requiring the agent to continuously re-explore the environment from scratch.

On a relatively high-dimensional continuous control task such as `HalfCheetah-v5`, which demands precise coordination of six actuated joints to produce stable forward locomotion, 10^6 timesteps proves insufficient for PPO to converge to a competitive policy. In contrast to Fujimoto *et al.* [3], who report TD3 as the top-performing algorithm on `HalfCheetah`, our experiments show SAC achieving a higher final mean reward compared to TD3. Importantly, unlike in preliminary experiments, PPO in this run did not collapse, where its learning curve shows a steady positive trend, confirming that the algorithm is learning, albeit slowly.

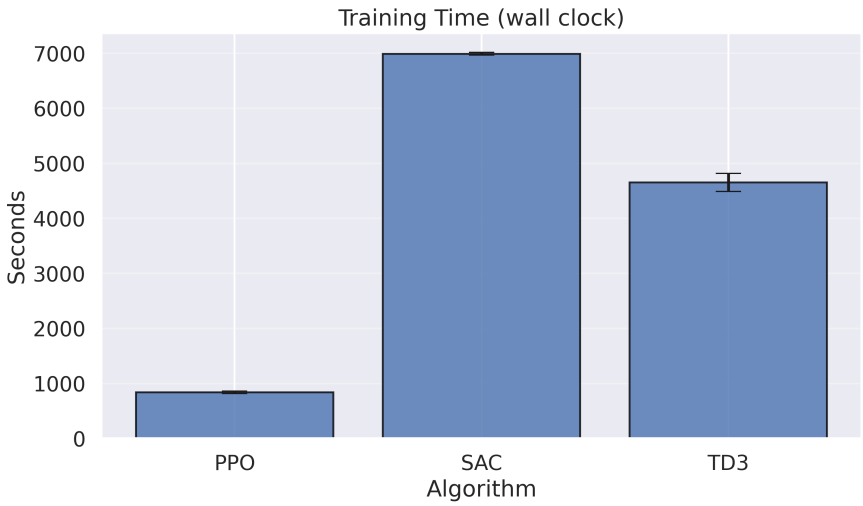

Figure 3: Wall-clock training time per algorithm (mean ± std across seeds).

4.2 PPO Variance and Sensitivity to Initialization

PPO exhibited notably high variance across seeds (std = 1,450), with seed 42 achieving more than 2.5× the reward of seeds 123 and 456. This sensitivity to random initialization is a known characteristic of on-policy gradient methods, where the quality of early rollouts can strongly bias the trajectory of subsequent updates. Without entropy regularization (`ent_coef=0.0`), the policy may prematurely converge to locally consistent but suboptimal behaviors. The RL Zoo tuned configuration for `HalfCheetah-v4` assigns a small but non-zero entropy coefficient (`ent_coef`$\approx 4 \times 10^{-4}$), suggesting that even minimal entropy regularization is beneficial for stabilizing PPO on this task [1]. Furthermore, a preliminary experiment applying `VecNormalize` exclusively to PPO resulted in a mean reward of ~ -150, indicating that observation normalization alone was insufficient to stabilize on-policy learning on this task.

4.3 SAC vs TD3: The Role of Entropy Regularization

Both off-policy algorithms converged to high-performing policies, but SAC consistently outperformed TD3 throughout training, achieving a final mean reward of 12,198.7 compared to TD3's 11,014.8, with a margin of approximately 10%. This advantage can be attributed to SAC's maximum entropy objective, which explicitly encourages the policy to remain stochastic and exploratory during training. On `HalfCheetah-v5`, whose reward landscape contains multiple locally optimal gaits, this exploratory pressure allows SAC to discover higher-reward locomotion strategies that a more deterministic policy such as TD3 may not reach.

TD3, however, demonstrated comparable consistency: its standard deviation (495) was similar to SAC's (511), and it converged faster in terms of wall-clock time. This

suggests that while SAC has a slight performance edge, TD3 is a strong alternative when compute efficiency is a priority.

4.4 Computational Cost vs Performance Trade-off

From a practical standpoint, the training time results reveal an informative Pareto trade-off. In multi-objective settings, a Pareto-optimal solution is one where no objective can be improved without degrading another. In this study, no single algorithm simultaneously achieves the highest reward and the lowest training cost. PPO was the fastest by a wide margin ($\sim$14 min), but delivered a policy far below the off-policy methods. TD3 required $\sim$78 min and achieved $\sim$90% of SAC's final reward. SAC required $\sim$117 min, approximately 1.5$\times$ TD3's cost, for a gain of $\sim$1,200 reward.

In resource-constrained settings, TD3 therefore represents the more computationally efficient choice, offering near-SAC performance at substantially lower wall-clock cost.

4.5 Limitations

This study is subject to several limitations. First, experiments were conducted on a single environment; generalizing conclusions to other MuJoCo tasks (e.g., `Ant-v5`, `Humanoid-v5`) requires additional evaluation. Second, hyperparameters were fixed to pre-tuned values from the RL Zoo and were not re-optimized for this specific experimental budget, which may disadvantage PPO in particular. Third, wall-clock training time comparisons are hardware-dependent and influenced by the asymmetric device assignment (PPO on CPU, SAC/TD3 on GPU). Fourth, with only 3 seeds per algorithm, statistical conclusions remain indicative rather than definitive.

5 Conclusion

This study evaluated PPO, SAC, and TD3 on `HalfCheetah-v5`, revealing clear trade-offs across sample efficiency, stability, and computational cost.

Off-policy methods overwhelmingly outperformed the on-policy baseline: both SAC and TD3 converged to high-reward locomotion policies within 10^6 timesteps, while PPO reached only $\sim$2,400 reward within the same budget, still showing an upward trend at the end of training. This reflects the core advantage of replay-buffer-based learning, which decouples data collection from optimization and reuses past experience across multiple gradient updates.

Between the two off-policy algorithms, the choice reduces to a performance-efficiency trade-off. SAC's maximum entropy objective drives broader exploration, yielding a $\sim$10% higher final reward, but at $\sim$1.5$\times$ the wall-clock cost of TD3. TD3, in turn, offers comparable cross-seed consistency and faster training, making it preferable when compute

constraints apply. Neither algorithm dominates unconditionally, the right choice depends on whether peak performance or resource efficiency is the priority.

Future work should extend comparisons to additional environments, re-evaluate algorithms with new hyperparameters and a larger timestep budget of 3 to 5×10^6 steps, increase the number of seeds to at least 5 to enable meaningful statistical testing, and explore deeper or wider network architectures to assess whether current rankings hold across varying model capacity.

References

[1] Zafarali Ahmed, Nicolas Le Roux, Mohammad Norouzi, and Dale Schuurmans. Understanding the impact of entropy on policy optimization. In *International conference on machine learning*, pages 151–160. PMLR, 2019.

[2] Marcin Andrychowicz, Anton Raichuk, Piotr Stańczyk, Manu Orsini, Sertan Girgin, Raphaël Marinier, Leonard Hussenot, Matthieu Geist, Olivier Pietquin, Marcin Michalski, et al. What matters for on-policy deep actor-critic methods? a large-scale study. In *International conference on learning representations (ICLR)*, 2021.

[3] Scott Fujimoto, Herke Hoof, and David Meger. Addressing function approximation error in actor-critic methods. In *International conference on machine learning*, pages 1587–1596. PMLR, 2018.

[4] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor, 2018.

[5] Timothy P Lillicrap, Jonathan J Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971*, 2015.

[6] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.